Accelerating Rainwater Flood Simulation with CUDA

Accelerator-Centric Computing Ecosystems – Programming Assignment Report

Jiawei Li
jiawei.li3@student.uva.nl
Computational Science, VU Amsterdam

Qin Min
mincheong82@gmail.com
Computer Science, VU Amsterdam

Mingjia Li
m.li14@student.vu.nl
Computer Science, VU Amsterdam

Abstract

At first built a baseline parallel version with three kernels per minute and a cell-per-thread mapping, which already ran up to 10.7× faster than the sequential reference.
Then applied four targeted optimizations: a per-cloud rainfall kernel over the cloud's bounding box, a gather-based spillage apply that removes the scatter scratch buffer, pinned-memory asynchronous readback of the stopping condition, and a GPU-side final reduction that skips copying the full water grid back to the host.
On DAS-5 GPU nodes the optimized version reaches up to 85× speedup for `large_mountains.in` and total 80.8× across six datasets.

1 Introduction

Background. Rainwater flood simulation computes how water flows over a terrain after rain falls. The terrain is stored as a 2D grid, where each cell holds a ground height. Each simulated minute the clouds move and drop rain, and water flows from higher cells to lower neighbors. The loop ends when the flow is small enough or a time limit is reached.

Motivation. The key observation is that, inside one minute, every cell does the same small amount of work, and cells read from themselves and their four neighbors. A CPU has to walk through the grid cell by cell, but a GPU can update thousands of cells at the same time.

Organization.

- Section 2 describes our two parallel designs, a baseline version and an optimized version, and explains the four changes that close most of the gap to the sequential cost.
- Section 3 covers the host-side main loop.
- Section 4 reports the experimental setup and measures both versions on six test cases.
- Section 5 discusses limitations and future work.

2 Parallelization

2.1 What is parallel

The outer loop over simulated minutes is sequential, because each step needs the water levels produced by the previous step. Inside one step the algorithm has three phases, and each phase is a grid pass and cells are independent:

- **Rainfall.** For each cell, add the rain from all clouds whose disk covers it. Cells only write to themselves.
- **Spillage compute.** For each cell with water, look at its four neighbors and decide how much water flows to each. Cells only read from their neighbors.
- **Spillage apply.** Each cell subtracts the losed water and adds the gained water. Cells only write to themselves.

The compute and apply phases must not overlap: all cells must finish computing the spillage before any cell updates its water level, otherwise the result will depends on the thread order.

2.2 Geometry

We map one CUDA thread to one cell. The block size is $16 \times 16 = 256$ threads. This is big enough to hide memory latency, small enough to fit a shared-memory reduction buffer of 256 values, and matches the warp size well ($256/32 = 8$ warps per block). Threads in the same row of a block access contiguous columns of the grid, which keeps global-memory reads contiguous.

2.3 Baseline parallel version

Our first CUDA version launches three kernels per minute over a full-grid block grid:

- **Rainfall kernel.** Each of the $R \times C$ threads loops over every cloud, computes the bounding box, tests whether its cell falls inside the cloud disk, and adds the rainfall.
- **Spillage compute.** Each thread looks at its four neighbors, computes how much water should flow to each one, and writes these four amounts into a scratch array `spillage_from_neigh`. The array has four slots per cell, one per direction (up, down, left, right). This is a *scatter* pattern: each thread writes into its neighbors' slots, not its own.
- **Spillage apply.** Each thread subtracts own outflow and sums the four incoming amounts from four slots of `spillage_from_neigh`, then clears the scratch arrays.

The stopping condition is read back every minute with a synchronous device-to-host copy. At the end , the full water grid is copied back to the host and the CPU iterates and compute the total-water and max-water-scenario statistics. The block-level shared-memory reductions for total rain, total water loss and the per-step maximum spillage are already in this baseline; they cut the number of global atomics by 256×, following the classic tree-reduction pattern [2].

This baseline reaches 3.3×–10.7× speedup (see Table 1). Three problems are visible in a profiler: (i) on large grids most cells in the rainfall kernel are idle because only a small fraction of cells is inside any cloud disk, but every thread still pays the cloud-loop bounding-box test; (ii) the scatter array is $4 \times n_{\text{cells}}$ floats and dominates memory traffic on large scenarios; (iii) the synchronous readback of the stopping condition blocks the host every minute, and the final full-grid copy from device to host wastes bandwidth especially for the largest datasets.

2.4 Optimization : four targeted improvements

(1) Per-cloud rainfall kernel. Instead of one big rainfall kernel where every grid thread iterates over all clouds, we launch one kernel per cloud, and the grid is sized to the cloud's bounding box. The cloud struct is passed by value as a kernel parameter, so it lives in constant memory for free. Threads no longer idle-iterate over clouds that do not cover them, and on large grids the per-cloud launch grid is much smaller than $R \times C$. This matches the workload-granularity guidance in [3] and removes a huge amount of wasted work on `large_mountains` in particular.

(2) Gather-based spillage apply. Converting scatter-style stencil updates into gather-style ones is a well-known optimization for stencil kernels on GPUs [1]. We replace the scatter scratch array `spillage_from_neigh` of size $4 \times n_{\text{cells}}$ with three per-cell arrays: `outflow_level`, `outflow_proportion` and `cell_height` (size n_{cells} each). The compute kernel writes only into its own cell. The apply kernel then reads its own outflow and *gathers* the inflow by reading the four neighbors' `outflow_proportion` and `cell_height`. This is race-free because `cell_height` is a snapshot taken before any water-level update. The change

- cuts the scratch memory from $4n$ floats to $3n$ floats,
- converts 4 random-pattern scatter writes per cell into 4 aligned reads per cell, which coalesce much better, and
- lets us drop the per-minute reset of the scratch array.

(3) Pinned-memory asynchronous readback with kernel overlap. The stopping condition needs a 4-byte maximum-spillage value back on the host each minute. In the baseline this was a blocking copy. We now (i) allocate the host buffer as pinned memory (`cudaHostAlloc`) so the copy uses DMA, (ii) queue the apply kernel immediately after the compute kernel, and (iii) issue an async copy for the max-spill value. The apply kernel runs while the 4-byte copy is in flight, and we only synchronize once at the end of the minute. This follows the overlap-copy-with-compute recipe from NVIDIA's best-practices guide [4].

(4) GPU-side final reduction. In the baseline the whole water-level array (for the largest case, 6.3M ints, ≈ 24 MB) was copied back to the host to compute the total-water and max-water statistics. We replace this with one extra kernel that does a block-level tree reduction [2] on the GPU and writes out only two scalars. The 24 MB transfer becomes 8 bytes.

We also keep the optimizations already present in the baseline: block-level shared-memory reductions, the `__float_as_int` trick for a float `atomicMax`, one-shot upload of the ground heightmap, and persistent device buffers.

3 Implementation details

The host main loop of the optimized version runs, per minute:

(1) Update cloud positions on the host (tens of clouds; not worth offloading).
(2) For each cloud, compute its bounding box on the host and launch `kernel_rain_cloud` with a grid sized to that bounding box. The cloud is passed by value.
(3) Reset the per-minute max-spillage scalar with an async memset, then launch `kernel_spillage_compute` over the full grid. It writes `outflow_level`, `outflow_proportion`, `cell_height` per cell, and contributes to the per-block reductions for `total_water_loss` and the maximum spillage.
(4) Launch `kernel_spillage_apply` over the full grid, before the max-spill readback, so its execution overlaps with the copy.
(5) `cudaMemcpyAsync` the max-spill scalar into pinned host memory, then `cudaStreamSynchronize` once, and decide whether to stop.

After the main loop, the final-stats kernel reduces the total water and the max water scenario on the GPU, and only three 8-byte totals plus two 4-byte scalars are copied back to the host. The water grid never leaves the device unless the `-DDEBUG` flag is set.

4 Evaluation

4.1 Experimental setup

All experiments were driven by our own `run_all_tests.sh` script, which iterates over every `.in` file in `test_files/`, runs both `flood_seq` and `flood_cuda` on a DAS-5 GPU compute node, invokes `check_correctness.py` on each pair of outputs, and prints a per-test and aggregate speedup table.

- **GPU:** NVIDIA TitanX on the DAS-5 gpunode.
- **CPU (baseline):** Intel Xeon on the same node.
- **CUDA toolkit:** 12.6 (`module load cuda12.6/toolkit`).
- **Compilers:** `nvcc` and `gcc`, both with `-O3`.

The metric is the wall-clock simulation time printed by the program (initialization excluded). For every test case the script runs the sequential and CUDA binaries on the same node with identical arguments, then extracts the `Time:` line from each run's output and computes the speedup. Correctness is checked automatically by `check_correctness.py`, which compares the seven result statistics against the sequential reference; all six cases pass in both CUDA versions.

4.2 Baseline vs optimized

Table 1 lists the runtimes and speedups of the baseline CUDA version and the optimized version, on the six test cases. Figure 1 plots the two speedup series side by side.

Table 1: Runtime (seconds) on DAS-5. "Base" is the three-kernel baseline of Section 2.3; "Opt." is the optimized version of Section 2.4.

Test	Seq	Base	Speed	Opt.	Speed
debug (90×90)	0.084	0.023	3.7×	0.014	6.0×
small (100×100)	0.045	0.014	3.3×	0.015	3.0×
custom (120×120)	0.344	0.036	9.7×	0.044	9.5×
med-hi (256×256)	1.004	0.106	9.5×	0.036	28.2×
med-lo (400×300)	0.975	0.091	10.7×	0.036	27.8×
large (3072×2048)	180.41	21.42	8.4×	2.21	85.0×
Total	182.87	21.68	**8.4×**	2.35	**80.8×**

4.3 Discussion of results

The optimized version is faster than the baseline on four of six tests, and dramatically so on the medium and large grids: 9.5 × →28.2×

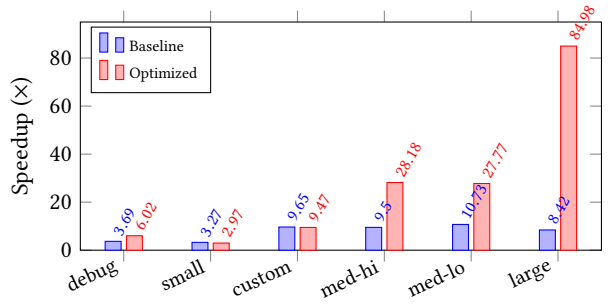

Figure 1: Speedup over the sequential baseline: baseline CUDA version vs optimized version.

on `med-hi`, 10.7 × →27.8× on `med-lo`, and 8.4 × →85.0× on `large`. The large case drops from 21.4 s to 2.2 s.

Two tests regress slightly. The `small` case (100×100, 16 clouds, 200 minutes) falls from 3.3× to 3.0×, and `custom` (120×120, 8 clouds, 500 minutes) from 9.7× to 9.5×. This is expected: optimization (1) replaces *one* rainfall kernel per minute with one launch *per cloud per minute*. On a tiny grid the fixed kernel-launch cost outweighs the saved idle work, so we pay more launches for little useful work. Once the grid is larger than a few hundred cells on a side, the saved work dominates.

The `large_mountains` result shows where each optimization pays off:

- The per-cloud rainfall kernel turns a full-grid launch that mostly does nothing into tens of small focused launches; on a 6.3M-cell grid this is the biggest single win.
- The gather-based apply shrinks the scratch array from 24 MB (4·6.3M floats) to 12 MB and converts scatter writes into coalesced reads, which is far friendlier to the Maxwell memory subsystem.
- The GPU-side final reduction saves a 24 MB device-to-host copy at the end of the simulation.
- Asynchronous readback hides the per-minute 4-byte copy behind the apply kernel, which is a small but consistent gain over 800 minutes.

The "Check precision loss" reported by each run is within the tolerance of `check_correctness.py` (e.g., 0.81 on debug, 14 on the medium dam cases, 2510 on `large_mountains` where the water totals are in the millions). All six tests pass in the optimized version.

5 Discussion and Conclusion

5.1 Challenges

The trickiest change was moving from scatter to gather in the spillage apply. In the baseline, each cell writes into four neighbor-indexed slots, and the apply kernel reads its own four slots – easy to reason about but expensive in memory traffic. In the gather version the apply kernel reads four neighbors' heights and proportions, so we had to be sure that these neighbor reads are taken from a *snapshot* of the heights (`cell_height`) instead of from `water_level`, which is updated in place. Getting this right while keeping the semantics identical to the reference took the most debugging time.

Building a float `atomicMax` was another small challenge, because CUDA only ships an integer version. The `__float_as_int` bit-cast works because all maxima are non-negative.

5.2 Limitations and possible optimizations

With 85× on the largest input we already match the "good implementation" bar from the README. Further gains are possible but smaller:

- Tile the ground and cell-height arrays into shared memory with a one-cell halo, so the four neighbor reads in the apply kernel hit shared memory instead of L1/L2.
- Fuse the compute and apply kernels using cooperative groups or a grid-wide barrier, saving one launch per minute.

5.3 Conclusion

Starting from a straightforward cell-per-thread baseline, four targeted changes – per-cloud rainfall kernels, gather-based spillage apply, pinned-memory asynchronous readback, and a GPU-side final reduction – push the total speedup over the sequential reference from 8.4× to 80.8×, and to 85× on the largest scenario. The biggest single win is the gather-based apply on large grids, which turns random-pattern scatter writes into coalesced reads and halves the scratch-memory footprint.

Time sheet (approximate)

Task	Hours
Reading the assignment PDF and project structure	2 h
Planning a ToDo list of the assignment	0.5 h
Reading the sequential code and understanding the algorithm	2 h
Designing the baseline parallelization strategy	1 h
Implementing the rainfall, spillage-compute and spillage-apply kernels	14 h
Testing and debugging	6 h
Designing and implementing the four optimizations	6 h
Running experiments on DAS-5 (baseline and optimized)	3 h
Writing the report	6 h
Total	**40.5 h**

References

[1] Kaushik Datta, Mark Murphy, Vasily Volkov, Samuel Williams, Jonathan Carter, Leonid Oliker, David Patterson, John Shalf, and Katherine Yelick. 2008. Stencil Computation Optimization and Auto-tuning on State-of-the-Art Multicore Architectures. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing (SC '08)*. IEEE Press, 1–12.

[2] Mark Harris. 2007. *Optimizing Parallel Reduction in CUDA*. NVIDIA Developer Technology Whitepaper. NVIDIA Corporation.

[3] David B. Kirk and Wen-mei W. Hwu. 2016. *Programming Massively Parallel Processors: A Hands-on Approach* (3rd ed.). Morgan Kaufmann.

[4] NVIDIA Corporation 2024. *CUDA C++ Best Practices Guide*. NVIDIA Corporation. Version 12.6.